Extending the Specification of the Toy Calculator

Big Step (Natural, Kahn-style) Operational Semantics

We seem to have bypassed a stage in our development — the relational (input-output) operational *specification* of the calculator. Let us extend the previous specification which we had given in Kahn's Natural semantics style to deal with the new forms that we have introduced. Recall that the calculator specifies a program that operates on abstract syntactic forms, returning canonical syntactic answers. We assume you are familiar with the presentation of inference rules with provisos, and will not provide any commentary with the rules.

(CalcNum) $\dfrac{}{\underline{n} \Longrightarrow \underline{n}}$

(CalcBool) $\dfrac{}{\underline{b} \Longrightarrow \underline{b}}$

(CalcPlus) $\dfrac{\underline{e_1} \Longrightarrow \underline{n_1} \quad \underline{e_2} \Longrightarrow \underline{n_2}}{\underline{e_1 + e_2} \Longrightarrow \underline{n}}$ provided $pLUS(\underline{n_1}, \underline{n_2}, \underline{n})$

(CalcTimes) $\dfrac{\underline{e_1} \Longrightarrow \underline{n_1} \quad \underline{e_2} \Longrightarrow \underline{n_2}}{\underline{e_1 * e_2} \Longrightarrow \underline{n}}$ provided $tIMES(\underline{n_1}, \underline{n_2}, \underline{n})$

(CalcNot) $\dfrac{\underline{e_1} \Longrightarrow \underline{b_1}}{\underline{\neg e_1} \Longrightarrow \underline{b}}$ provided $myNot(\underline{b_1}, \underline{b})$

(CalcAnd) $\dfrac{\underline{e_1} \Longrightarrow \underline{b_1} \quad \underline{e_2} \Longrightarrow \underline{b_2}}{\underline{e_1 \wedge e_2} \Longrightarrow \underline{b}}$ provided $myAnd(\underline{b_1}, \underline{b_2}, \underline{b})$

CalcOr) $\dfrac{\underline{e_1} \Longrightarrow \underline{b_1} \quad \underline{e_2} \Longrightarrow \underline{b_2}}{\underline{e_1 \vee e_2} \Longrightarrow \underline{b}}$ provided $myOr(\underline{b_1}, \underline{b_2}, \underline{b})$

(CalcEq) $\dfrac{\underline{e_1} \Longrightarrow \underline{a_1} \quad \underline{e_2} \Longrightarrow \underline{a_2}}{\underline{e_1 = e_2} \Longrightarrow \underline{b}}$ provided $eQ(\underline{a_1}, \underline{a_2}, \underline{b})$

(CalcGt) $\dfrac{\underline{e_1} \Longrightarrow \underline{n_1} \quad \underline{e_2} \Longrightarrow \underline{n_2}}{\underline{e_1 > e_2} \Longrightarrow \underline{b}}$ provided $gT(\underline{n_1}, \underline{n_2}, \underline{b})$

Ensuring that Expressions are Well-formed

We have to ensure that the expressions which we provide to the calculator are not ill-formed, such as adding a numeric expression to a boolean, etc. The mechanism to do so

is to use a "type system", and specify a checker that weeds out the badly formed expressions.

Let us define (toy language) *types*, reflecting numeric and boolean expressions. Let us call these (unimaginatively) Int and Bool. So $\tau \in Typ = \{\text{Int}, \text{Bool}\}$. Note that these are just *syntactic names* for the types we are creating for *our toy language*. [As we introduce more types and type constructions to our toy language, we will introduce more toy language type names/type operations.]

We specify a *relation* $\vdash e : \tau$ to say that (according to the rules of the language) "*expression* e *has type* τ." The turnstile and the colon are just markers/separators, but we will expect you to adhere to this notation. This typing relation is recursively defined, and again we use inference rules that follow the structure of the expression e.

(NumT) $\dfrac{}{\vdash n : \text{Int}}$ All numerals n have type Int.

(BoolT) $\dfrac{}{\vdash b : \text{Bool}}$ All boolean (modelled in OCaml using type `myBool`) constants b have type Bool.

(PlusT) $\dfrac{\vdash e_1 : \text{Int} \quad \vdash e_2 : \text{Int}}{\vdash e_1 + e_2 : \text{Int}}$

All addition expressions $e_1 + e_2$ have type Int, provided the subexpressions e_1 and e_2 both have type Int

(TimesT) $\dfrac{\vdash e_1 : \text{Int} \quad \vdash e_2 : \text{Int}}{\vdash e_1 * e_2 : \text{Int}}$

All multiplication expressions $e_1 * e_2$ have type Int, provided the subexpressions e_1 and e_2 both have type Int

(NotT) $\dfrac{\vdash e_1 : \text{Bool}}{\vdash \neg e_1 : \text{Bool}}$

All negation expressions $\neg e_1$ have type Bool, provided the subexpressions e_1 have type Bool

(AndT) $\dfrac{\vdash e_1 : \text{Bool} \quad \vdash e_2 : \text{Bool}}{\vdash e_1 \wedge e_2 : \text{Bool}}$

All conjunction expressions $e_1 \wedge e_2$ have type Bool, provided the subexpressions e_1 and e_2 both have type Bool

(OrT) $\dfrac{\vdash e_1 : \text{Bool} \quad \vdash e_2 : \text{Bool}}{\vdash e_1 \vee e_2 : \text{Bool}}$

All disjunction expressions $e_1 \vee e_2$ have type Bool, provided the subexpressions e_1 and e_2 both have type Bool

$$\textbf{(EqT)}\ \frac{\vdash e_1 : \tau \qquad \vdash e_2 : \tau}{\vdash e_1 = e_2 : \text{Bool}}$$

All equality expressions $e_1 = e_2$ have type Bool, provided the subexpressions e_1 and e_2 both have the same type τ

$$\textbf{(GtT)}\ \frac{\vdash e_1 : \text{Int} \qquad \vdash e_2 : \text{Int}}{\vdash e_1 > e_2 : \text{Bool}}$$

All greater-than expressions $e_1 > e_2$ have type Bool, provided the subexpressions e_1 and e_2 both have type Int

Expressions that can be given a type under these rules are called "well-typed". Otherwise the expression is ill-formed, and so does not have a type ("ill-typed").

Type Preservation (version 1)

One of the nicest results in Programming Language theory is the following:

> **Theorem** (Type Preservation under $e \Longrightarrow a$)
> For all expressions $e \in Exp$, for all answers $a \in Ans$, for all types $\tau \in Typ$, if $\vdash e : \tau$ and $e \Longrightarrow a$, then $\vdash a : \tau$

The importance of this result is that calculation does not change the type of a well-typed expression — the answer is of the same type. So we can type-check the expression once and for all (statically/ at compile time / before execution), and do not have to worry that a well-typed expression will become ill-typed any time during execution. From a pragmatic viewpoint, this means we need not insert (expensive) type checks in the abstract machine code during compilation just to ensure that the expression is well-typed.

> [Note however that some checks may still have to be inserted. For example, the typing rules do not prevent division by zero].

A crucial assumption we will make is that the elementary operations are type-sound. For example, addition of two numerals returns a numeral and e.g., $gT(n_1, n_2, b)$ will return a boolean b (i.e., $\vdash b : \text{Bool}$) for every pair of numerals n_1, n_2 (i.e., $\vdash n_1 : \text{Int}$ and $\vdash n_2 : \text{Int}$.)

Guess how we will prove this theorem? How are the relations $\vdash e : \tau$ and $e \Longrightarrow a$ defined? By induction on the structure of expressions $e \in Exp$ — so structural induction is the technique that is applicable.

Proof (By Induction on the structure/*ht* of e).
Base cases ($ht(e) = 0$)
Subcase ($e \equiv n$)

Assume $\vdash \underline{e} : \underline{\tau}$. Therefore $\underline{\tau} = \underline{\text{Int}}$. Now $\underline{n} \Longrightarrow \underline{n}$. Trivially $\vdash \underline{n} : \underline{\text{Int}}$

Subcase ($\underline{e} \equiv \underline{b}$)
Assume $\vdash \underline{e} : \underline{\tau}$. Therefore $\underline{\tau} = \underline{\text{Bool}}$. Now $\underline{b} \Longrightarrow \underline{b}$. Trivially $\vdash \underline{b} : \underline{\text{Bool}}$

Induction Hypothesis: Assume that for all expressions $\underline{e'}$, such that $ht(\underline{e'}) \leq k$, for all answers $\underline{a'}$, for all types $\underline{\tau'}$, if $\vdash \underline{e'} : \underline{\tau'}$ and $\underline{e'} \Longrightarrow \underline{a'}$, then $\vdash \underline{a'} : \underline{\tau'}$

Induction Step ($ht(\underline{e}) = 1 + k$)
Subcase ($\underline{e} \equiv \underline{e_1 > e_2}$)
Assume $\vdash \underline{e} : \underline{\tau}$. Therefore by **(GtT)**, $\underline{\tau} = \underline{\text{Bool}}$ and $\vdash \underline{e_1} : \underline{\text{Int}}$ and $\vdash \underline{e_2} : \underline{\text{Int}}$.
Now by **(CalcGt)** $\underline{e_1 > e_2} \Longrightarrow \underline{b}$ provided $\underline{e_1} \Longrightarrow \underline{n_1}$, $\underline{e_2} \Longrightarrow \underline{n_2}$ and $gT(\underline{n_1}, \underline{n_2}, \underline{b})$ for some $\underline{n_1}, \underline{n_2}$.
By IH on $\underline{e_1}$, $\vdash \underline{n_1} : \underline{\text{Int}}$ and by IH on $\underline{e_2}$, $\vdash \underline{n_2} : \underline{\text{Int}}$
So by *type-soundness* of $gT(\underline{n_1}, \underline{n_2}, \underline{b})$, we have $\vdash \underline{b} : \underline{\text{Bool}}$.

All other subcases are similar (and perhaps simpler). □

Exercise: Complete the proof of the Type Preservation Theorem (all other inductive subcases).

Exercise: Encode the type-checking relation $\vdash \underline{e} : \underline{\tau}$ in PROLOG as a predicate `hastype(E,T).`

A Rudimentary Type Checker in OCaml

Since we have at least two types, we want to avoid evaluating or compiling ill-typed expressions such as
`Plus(Bl T, Num 3)`

We can code a basic type-checker `type_of` that analyses an expression, and finds its type if it is well-formed, raising a type error otherwise.

Let us first declare that we have two different types in our toy language: *Int* and *Bool* (this set of types will grow later).

```
type types = Int | Bool;;
```

Let us also define an exception that is raised in case of a type error:

```
exception TypeError of exp * string;;
```

(the `string` component will let our type-checker give some more information about the expected type).

```
let rec type_of e = match e with
    Num n  -> Int
  | Bl b -> Bool
  | Plus (e1, e2)  ->    (* (Int * Int) -> Int *)
      if (type_of e1) = Int
      then
        (if (type_of e2) = Int then Int
```

```
          else raise (TypeError (e2, "expected to be of type
Int")))
      else raise (TypeError (e1, "expected to be of type Int"))
  | Times (e1, e2)  ->  (* (Int * Int) -> Int *)
      if (type_of e1) = Int
      then
        (if (type_of e2) = Int then Int
         else raise (TypeError (e2, "expected to be of type
Int")))
      else raise (TypeError (e1, "expected to be of type Int"))
  | And (e1, e2)  ->  (* (Bool * Bool) -> Bool *)
      if (type_of e1) = Bool
      then
        (if (type_of e2) = Bool then Bool
         else raise (TypeError (e2, "expected to be of type
Bool")))
      else raise (TypeError (e1, "expected to be of type Bool"))
  | Or (e1, e2)  -> (* (Bool * Bool) -> Bool *)
      if (type_of e1) = Bool
      then
        (if (type_of e2) = Bool then Bool
         else raise (TypeError (e2, "expected to be of type
Bool")))
      else raise (TypeError (e1, "expected to be of type Bool"))
  | Not e1 -> (* Bool -> Bool *)
         if (type_of e1) = Bool then Bool
         else raise (TypeError (e1, "expected to be of type
Bool"))
  | Eq (e1, e2)  -> (* (tau * tau) -> Bool *)
      if (type_of e1) = (type_of e2)
      then Bool
      else raise (TypeError (e, " subexpressions not of same
type"))
  | Gt(e1, e2)  -> (* (Int * Int) -> Bool *)
      if (type_of e1) = Int
      then
        (if (type_of e2) = Int  then Bool
         else raise (TypeError (e2, "expected to be of type
Int")))
      else raise (TypeError (e1, "expected to be of type Int"))
;;
```

Let us discuss just a few cases in the definition above:

- If the expression is a numeral, then its type is `Int`:
  ```
  Num n  -> Int
  ```
- If the expression is a boolean constant, then its type is `Bool`:
  ```
  | Bl b -> Bool
  ```
- If the expression is an arithmetic operation, then its type is `Int`, provided its subexpressions are also of type `Int`:
  ```
    | Plus (e1, e2)  ->   (* (Int * Int) -> Int *)
          if (type_of e1) = Int
  ```

```
            then
              (if (type_of e2) = Int then Int
               else raise (TypeError (e2, "expected to be of type
    Int")))
            else raise (TypeError (e1, "expected to be of type
    Int"))
```

- Likewise, if the expression is a boolean operation, then its type is `Bool`, provided its subexpressions are also of type `Bool` (not shown).
- If the expression is a comparison operation, then its type is `Bool`, provided both its subexpressions are of type `Int`:

```
Gt(e1, e2)  -> (* (Int * Int) -> Bool *)
      if (type_of e1) = Int
      then
        (if (type_of e2) = Int  then Bool
         else raise (TypeError (e2, "expected to be of type
Int")))
      else raise (TypeError (e1, "expected to be of type Int"))
```

You can check that the following ill-typed expression will raise a type error:

```
type_of (Plus(Bl T, Num 3));;
```

An expression (program) is *well-typed* if it has a type (and a type error exception is not raised).

The main slogan (an important theorem for strongly typed languages) we would like to show is:

> *"Well-typed programs do not go (raise)* `Wrong`*."*

This property of well-typed programs allows us to dispense with type-checking at run-time, and therefore more efficient code.

If a program is *not well-typed*, we should not even bother with trying to evaluate it, or compile it. The interesting thing about strongly typed languages is that their insistence on programs being well-typed, while viewed as an inconvenience by many coders, forces the programmer to be careful when writing the program— a surprisingly large fraction of logical errors get flagged as type errors.